EXPERIMENT No.: 05

Aim of Experiment: To implement a closed loop control system.

Objectives:
To implement a **closed-loop speed control system** for a DC motor using an encoder as the feedback sensor and Arduino as the controller.

Tasks
1. Measure the rotational speed of the DC motor using an encoder.
2. Compare the measured/actual speed with the reference (set-point) speed.
3. Calculate the speed error.
4. Adjust the PWM control signal supplied to the motor driver.
5. Maintain the motor speed close to the desired reference speed.

Basic Theory

3.1 Closed-Loop Control System

A closed-loop control system continuously measures the actual output and compares it with the desired reference input.
For the present experiment:
Reference Speed → Comparator → Controller → Motor Driver → DC Motor → Actual Speed
The encoder provides feedback from the motor to the Arduino.
The basic control error is:

$$e(t) = R(t) - Y(t)$$

where:

- $R(t)$ = reference/set-point speed
- $Y(t)$ = actual measured motor speed
- $e(t)$ = speed error

The Arduino processes this error and changes the PWM duty cycle supplied to the motor driver.

3.2 Encoder

An encoder converts the rotational motion of the motor shaft into electrical pulses.
For an encoder having N pulses per revolution:

$$RPM = \frac{P}{N} \times \frac{60}{T}$$

where:

- P = number of encoder pulses counted
- N = pulses per revolution
- T = measurement time in seconds

For example, if the encoder produces 100 pulses/revolution and 50 pulses are counted in 0.5 s:

$$RPM = \frac{50}{100} \times \frac{60}{0.5} = 60\ RPM$$

3.3 PWM Control

Arduino generates a PWM signal to control the average voltage supplied to the motor.
PWM duty cycle is:

$$D = \frac{T_{ON}}{T} \times 100$$

A higher PWM duty cycle generally produces a higher motor speed, although the exact relationship depends on the motor, load and driver.

3.4 PID Control

A PID controller can be represented by:

$$u(t) = K_P e(t) + K_I \int e(t)dt + K_D \frac{de(t)}{dt}$$

where:

- K_P = proportional gain
- K_I = integral gain
- K_D = derivative gain
- $u(t)$ = control signal

For a speed-control experiment, a **PI controller** is often sufficient because it reduces steady-state speed error without requiring derivative action.

Components Required:

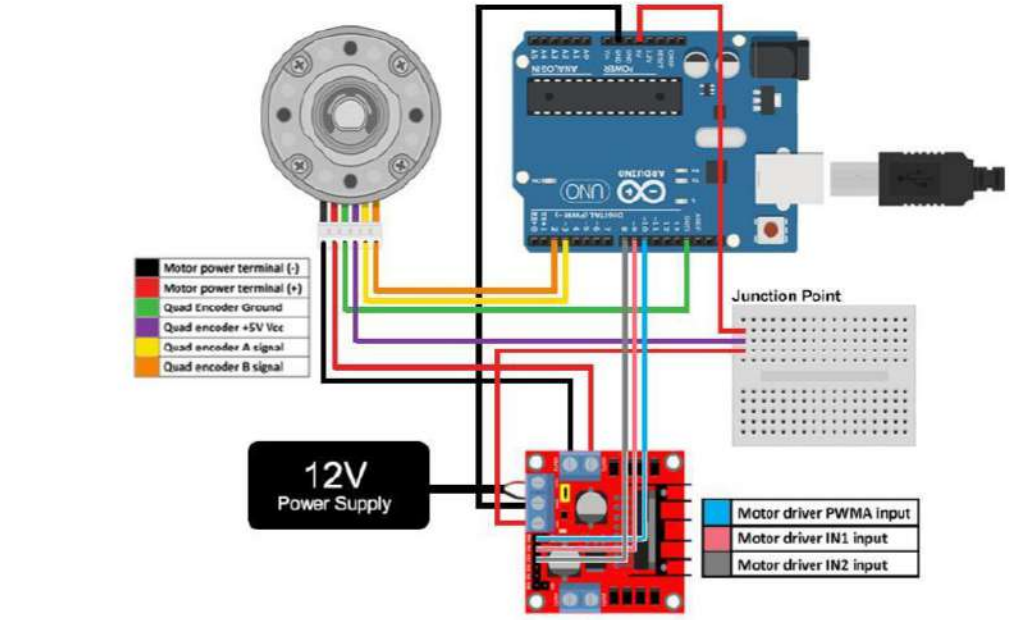
Sr. No.	Component Specification	Quantity
1	Arduino Uno	1
2	DC geared motor with encoder	1
3	Motor driver - L298N or equivalent	1
4	DC power supply As required by motor	1
5	Connecting wires	As required
6	USB cable	1
7	Computer/Laptop	1
8	Breadboard/terminal connections	As required

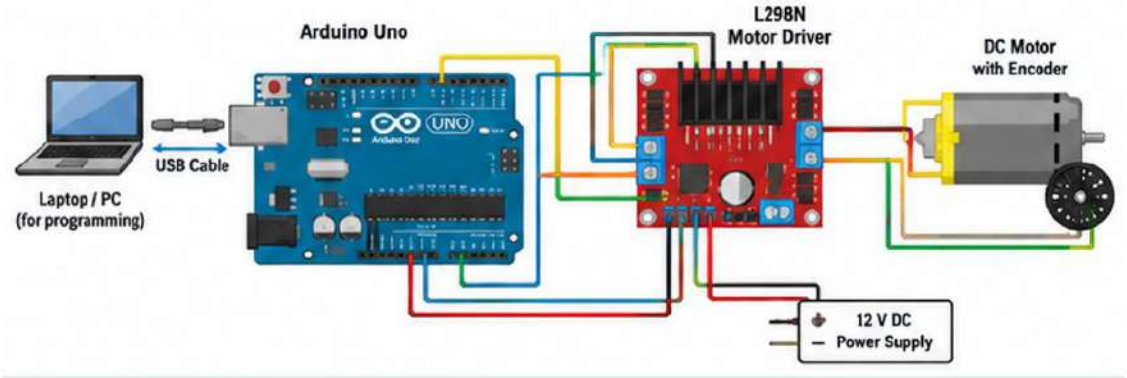
Key Connections

The following is an example connection using Arduino Uno + L298N motor driver + quadrature encoder.

Arduino Pin	Connected To	Purpose
D2	Encoder A	Encoder pulse input
D3	Encoder B	Encoder direction/second channel
D7	L298N IN2	Motor direction
D8	L298N IN1	Motor direction
D9 (PWM)	L298N ENA	Motor speed control
5 V	Encoder VCC*	Encoder supply

GND	Encoder GND + Driver GND	Common ground
Motor terminals	L298N OUT1/OUT2	Motor connection
External DC supply	L298N motor supply	Motor power





Program :

```
#include <Encoder.h>
#include <PID_v1.h>

// ======================================================
// CLOSED LOOP SPEED CONTROL OF DC MOTOR
// Arduino UNO + Encoder + L298N + PID Controller
// ======================================================

// ---------------- Encoder Pins ----------------
```

```
const int encA = 2;
const int encB = 3;

// ---------------- L298N Motor Driver Pins ----------------
const int pwmPin = 9;       // ENA - PWM
const int in1 = 8;          // IN1
const int in2 = 7;          // IN2

// ---------------- Encoder Specification ----------------
// Change this value according to your encoder.
//
// If the encoder is rated as 100 PPR per channel and
// the Encoder library counts all quadrature transitions,
// use 400 counts/revolution.
//
// If your encoder specification directly gives 100 counts
// per revolution as counted by the Arduino, use 100.
const double COUNTS_PER_REV = 400.0;

// ---------------- PID Variables ----------------
double Setpoint = 100.0;    // Desired speed in RPM
double Input = 0.0;         // Actual motor speed in RPM
double Output = 0.0;        // PWM output

// PID gains
double Kp = 1.0;
double Ki = 0.5;
double Kd = 0.1;

// Create PID controller
PID myPID(&Input, &Output, &Setpoint,
        Kp, Ki, Kd, DIRECT);

// Create encoder object
Encoder myEnc(encA, encB);

// ---------------- Timing ----------------
unsigned long lastTime = 0;
const unsigned long sampleTime = 100;   // 100 ms

// ====================================================
// SETUP
// ====================================================
void setup()
{
  Serial.begin(9600);

  // Motor driver pins
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
  pinMode(pwmPin, OUTPUT);

  // Motor direction - Forward
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);

  // Set PID limits
```

```
  myPID.SetOutputLimits(0, 255);

  // PID sample time
  myPID.SetSampleTime(sampleTime);

  // Start PID controller
  myPID.SetMode(AUTOMATIC);

  // Reset encoder count
  myEnc.write(0);

  lastTime = millis();

  Serial.println("Closed Loop DC Motor Speed Control");
  Serial.println("---------------------------------");
  Serial.println("Setpoint(RPM)  Actual(RPM)  PWM");
}

// ================================================
// MAIN LOOP
// ================================================
void loop()
{
  unsigned long now = millis();

  // Execute speed calculation and PID control
  // every 100 ms
  if (now - lastTime >= sampleTime)
  {
    // Calculate actual elapsed time
    double elapsedTime = now - lastTime;

    lastTime = now;

    // Read encoder count accumulated during
    // the previous sampling period
    long counts = myEnc.read();

    // Reset encoder count for the next sampling period
    myEnc.write(0);

    // ------------------------------------------------
    // Calculate Motor Speed in RPM
    // ------------------------------------------------
    //
    // RPM = Counts x 60000
    //       ----------------------------
    //       Counts/Rev x Time(ms)
    //
    Input = (abs(counts) * 60000.0) /
          (COUNTS_PER_REV * elapsedTime);

    // ------------------------------------------------
    // PID Calculation
    // ------------------------------------------------
    myPID.Compute();
```

```
    // ------------------------------------------------
    // Motor Direction and PWM Control
    // ------------------------------------------------

    // Forward rotation
    digitalWrite(in1, HIGH);
    digitalWrite(in2, LOW);

    // Apply PID output as PWM
    analogWrite(pwmPin,
            constrain((int)Output, 0, 255));

    // ------------------------------------------------
    // Display Results on Serial Monitor
    // ------------------------------------------------
    Serial.print("Setpoint: ");
    Serial.print(Setpoint);

    Serial.print(" RPM, Actual: ");
    Serial.print(Input);

    Serial.print(" RPM, PWM: ");
    Serial.println(Output);
  }
}
```

Experimental Procedure

Step 1 – Hardware Preparation

1. Collect Arduino Uno, DC motor with encoder, motor driver and power supply.
2. Check the voltage and current rating of the motor.
3. Keep the motor initially unloaded.

Step 2 – Make Connections

1. Connect encoder output A to Arduino D2.
2. Connect encoder output B to Arduino D3.
3. Connect the motor to the motor-driver output.
4. Connect the motor driver's direction inputs to Arduino D7 and D8.
5. Connect the PWM/Enable input to Arduino D9.
6. Connect the encoder supply according to its rating.
7. Connect Arduino ground and motor-driver ground together.

Step 3 – Upload Program

1. Open Arduino IDE.
2. Select the appropriate Arduino board and COM port.
3. Enter the encoder PPR value in the program.
4. Enter the desired reference speed.
5. Upload the program.

Step 4 – Start the System

1. Switch ON the motor power supply.
2. Open the Serial Monitor.
3. Observe the reference speed, actual speed, error and PWM value.
4. Allow the motor to reach approximately steady-state speed.

Step 5 – Change Reference Speed

Set different reference speeds, for example:

- 50 RPM
- 100 RPM
- 150 RPM

- 200 RPM
- 250 RPM

5. Record the actual motor speed and PWM value.
6. Calculate the speed error.
7. Repeat the experiment after changing the PI parameters if required.

Observation Table

Sr. No.	Reference Speed (RPM)	Actual Speed (RPM)	Error (RPM)	PWM (0–255)	Remarks
1	50				
2	100				
3	150				
4	200				
5	250				

Error Calculation

$$\text{Error} = \text{Reference Speed} - \text{Actual Speed}$$

Percentage error:

$$\% \text{ Error} = \frac{|\text{ Reference Speed} - \text{Actual Speed }|}{\text{Reference Speed}} \times 100$$

Additional observation table for controller tuning:

Trial	K_P	K_I	Reference RPM	Actual RPM	Error %	Response
1			100			
2			100			
3			150			
4			200			

Conclusion
